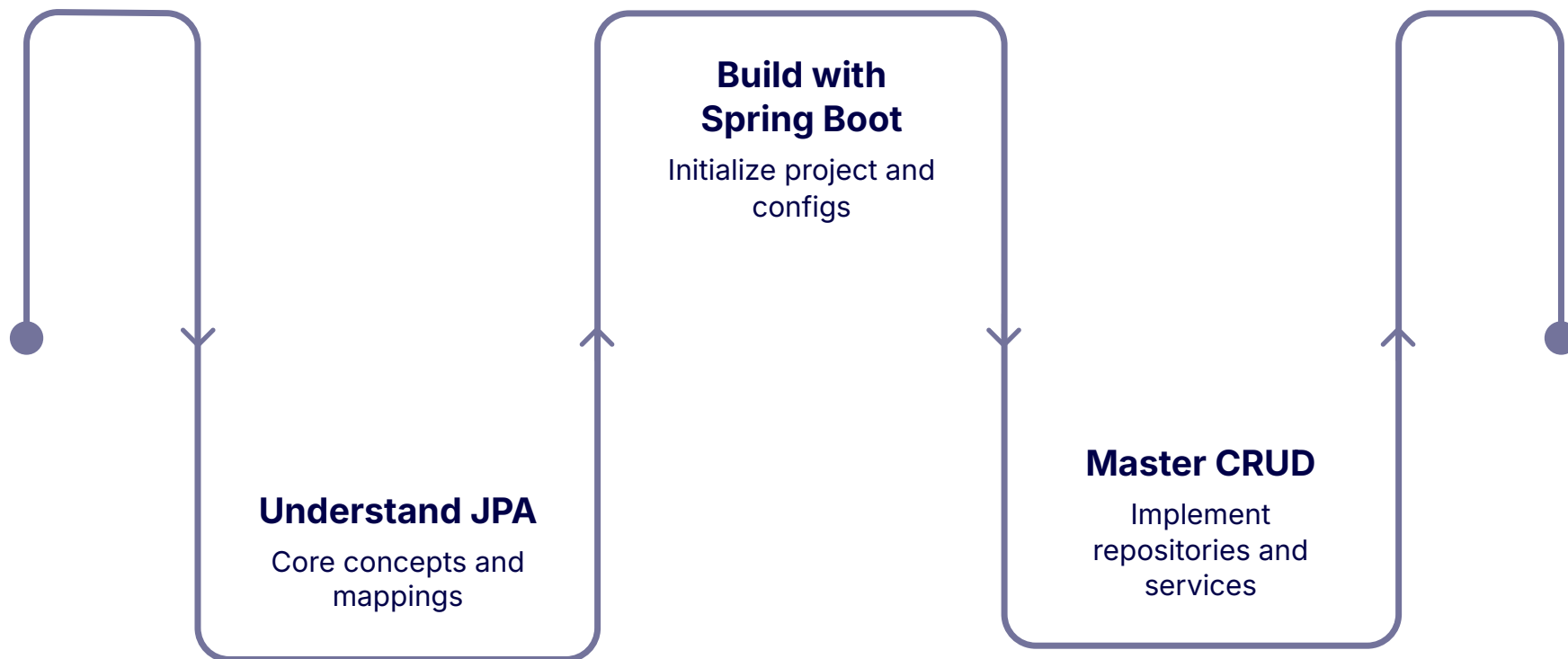


Spring Boot + JPA Basics



Naresh Chaurasia

What You Will Learn

By the end of this session, you will have a solid foundation in Spring Boot and JPA — ready to apply in real projects.

1

What is JPA

Understand the Java Persistence API and its role in modern development.

2

Why We Use JPA

Discover the problems JPA solves and why it matters.

3

Spring Boot + JPA

See how Spring Boot integrates JPA seamlessly.

4

CRUD Operations

Build create, read, update, and delete functionality from scratch.

The Problem *Before* JPA

Writing SQL Manually

Every query had to be hand-crafted, increasing the chance of syntax errors and inconsistencies.

Managing DB Connections

Opening, closing, and pooling connections was the developer's responsibility — tedious and fragile.

Mapping Data Manually

Converting ResultSet rows into Java objects required repetitive, boilerplate code.

The Core Issue

Too much focus on **database code** instead of **business logic**.

Developers spent the majority of their time wiring plumbing rather than solving real problems.



Time-consuming and highly error-prone

What is JPA?

JPA = Java Persistence API

A standard specification defined by Java EE for managing relational data in Java applications.

A Specification, Not a Tool

JPA defines *what* to do. Implementations like Hibernate define *how* to do it.

Java Objects ↔ Database

Interact with your database using plain Java objects — no raw SQL required.

The Big Idea

JPA bridges the gap between the **object-oriented world** of Java and the **relational world** of databases.

  Java Object ↔ Database Table — this mapping is at the heart of everything JPA does.

What is Spring Data JPA?

Spring Data JPA is a layer built **on top of JPA** that makes database access even simpler and faster to implement.



Built on Top of JPA

Spring Data JPA wraps the JPA specification and adds powerful abstractions, so you get all the benefits of JPA with far less effort.



Reduces Boilerplate Code

Gone are the days of writing repetitive data access code. Spring Data JPA handles the heavy lifting automatically.



Ready-Made Methods

Out of the box, you get `save()`, `findAll()`, `deleteById()` and many more — no SQL needed for basic operations.

✅ 👉 No need to write SQL for basic operations — Spring Data JPA does it for you.

Key Concept: ORM

Object Relational Mapping

ORM is the technique that allows JPA to translate between Java objects and database tables automatically.



  **Example:** The `User` table in your database maps directly to the `User` class in your Java code. Each column becomes a field, and each row becomes an object.

Project Structure

A typical Spring Boot JPA project follows a clean, layered architecture. Each layer has a clear responsibility.



Entity (Model)

Represents a database table as a Java class. Annotated with `@Entity`.



Service (Optional)

Contains business logic. Sits between the controller and repository.



Repository

Interface that provides CRUD methods. Extends `JpaRepository`.



Controller

Handles HTTP requests and responses. Exposes REST endpoints.

Entity Class

Represents a Database Table

```
@Entity
public class User {

    @Id
    private Long id;

    private String name;

    private String email;

}
```

Key Annotations

@Entity

Tells JPA this class maps to a database table.

@Id

Marks the primary key field of the table.

✅ 👉 One object = one row in the database

Repository Interface

```
public interface UserRepository  
    extends JpaRepository<User, Long> {  
  
}
```

That's it — just extend `JpaRepository` and Spring does the rest. No implementation class needed.

Spring Provides Out of the Box

save()

Insert or update a record

findAll()

Retrieve all records

delete()

Remove a record

CRUD Operations

Spring Data JPA maps all four fundamental database operations to simple, intuitive method calls.

Create

`save(entity)`



Read

`findById()`

`findAll()`

Update

`save(entity)`



Delete

`deleteById(id)`

📌 Note: Both **Create** and **Update** use `save()`. JPA determines whether to INSERT or UPDATE based on whether the entity already has an ID.

Controller Example

The controller exposes your data operations as HTTP endpoints. Here is a simple example saving a user via a POST request.

```
@RestController
public class UserController {

    @Autowired
    private UserRepository repo;

    @PostMapping("/users")
    public User saveUser(
        @RequestBody User user) {

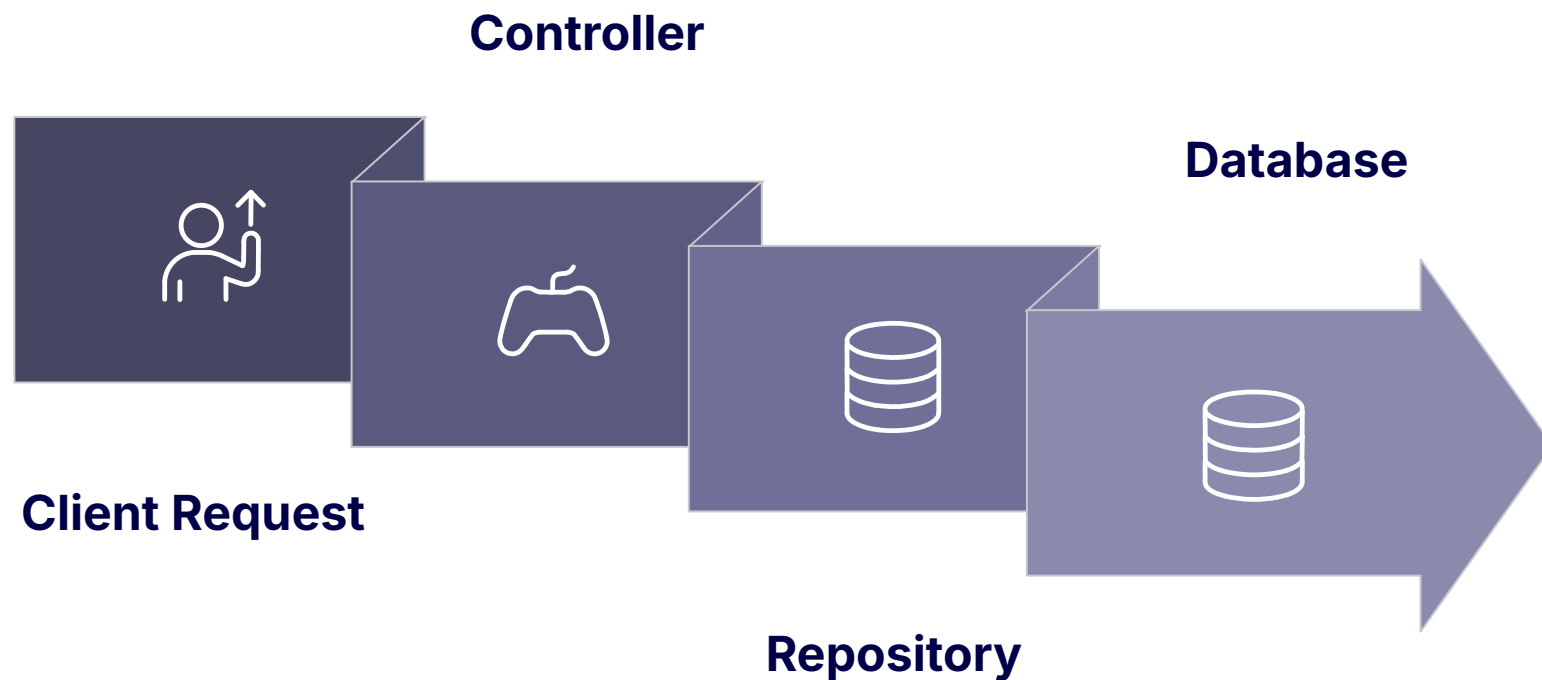
        return repo.save(user);
    }
}
```

Annotations Explained

- `@RestController` — marks this as a REST API controller
- `@Autowired` — injects the repository automatically
- `@PostMapping` — maps HTTP POST to this method
- `@RequestBody` — reads JSON from the request body

Request Flow

Understanding how a request travels through your application is essential. The flow is clean, linear, and predictable.



✅ 👉 **No SQL required.** The entire flow from HTTP request to database response is handled by Spring Boot and JPA — you just wire the pieces together.

Example Use Case

Employee Management System

Let's put it all together with a real-world scenario. An Employee Management System is a classic example that demonstrates all four CRUD operations in action.



Add Employee

Use `POST /employees` with a JSON body. The controller calls `repo.save()` to persist the new record.



View Employees

Use `GET /employees` to retrieve all records. Spring calls `repo.findAll()` and returns the list.



Update Employee

Use `PUT /employees/{id}` to modify an existing record. JPA detects the existing ID and issues an `UPDATE`.



Delete Employee

Use `DELETE /employees/{id}` to remove a record. The repository calls `deleteById()`.

Advantages

Spring Data JPA delivers tangible benefits that directly impact your productivity and the quality of your codebase.



Less Code

Eliminate hundreds of lines of boilerplate. No more manual SQL, no more ResultSet mapping — just clean, concise Java.



Faster Development

Get features built and shipped quicker. The repository pattern means your data layer is ready in minutes, not hours.



Easy Maintenance

Standardised patterns make it easy for any team member to understand, modify, and extend the codebase confidently.



Clean Architecture

Separation of concerns is built in. Each layer has a clear role, making your application easier to test and scale.

Common Mistakes

New developers frequently encounter the same pitfalls. Being aware of these early will save you hours of debugging.

✗ Missing @Entity

Without this annotation, JPA does not recognise your class as a mapped table. You will get runtime errors when the application starts.

✗ Missing @Id

Every entity must have a primary key field annotated with @Id. Omitting it causes JPA to throw a mapping exception.

✗ Wrong Data Types

Mismatched types between your Java field and the database column (e.g. using `int` instead of `Long` for IDs) can cause subtle bugs.

✗ Not Understanding Relationships

Jumping into @OneToMany or @ManyToOne without understanding them leads to N+1 query problems and unexpected behaviour.

Relationships (Intro)

Once you are comfortable with basic CRUD, the next step is understanding how entities relate to each other — just as tables do in a relational database.

OneToOne

One entity is associated with exactly one other entity.

Example: One User → One Profile

OneToMany

One entity is associated with multiple instances of another.

Example: One User → Many Orders

ManyToOne

Many entities are associated with one parent entity.

Example: Many Orders → One User

  **Key Takeaway:** Relationships are how you model real-world connections in your data. Master the basics first, then explore @ManyToMany and fetch strategies.

Industry Usage

Spring Data JPA is not just a learning exercise — it is a core technology used across the industry every day.

Where You Will See It

→ Real Projects

Widely adopted in enterprise and startup applications alike.

→ Spring Boot Apps

The de facto standard for data access in the Spring ecosystem.

→ Microservices

Each microservice typically owns its own database, accessed via JPA.

✅ 👉 **You will use this in your job.** Learning Spring Data JPA now means you are immediately productive on real-world projects from day one.

Whether you are building a monolith or a distributed microservices architecture, Spring Data JPA is the tool teams reach for when they need reliable, maintainable database access in Java.

Hands-On Task

Build a Simple Application

Now it is your turn. Apply everything you have learned by building a small but complete Spring Boot JPA application from scratch.

01

Create the User Entity

Define a `User` class with `@Entity`, `@Id`, and fields for `id`, `name`, and `email`.

03

Get All Users

Implement a `GET /users` endpoint that returns all users from the database using `findAll()`.

02

Build the Create API

Implement a `POST /users` endpoint in your controller that accepts a JSON body and saves the user.

04

Delete a User

Implement a `DELETE /users/{id}` endpoint that removes a user by their ID using `deleteById()`.

📝 Test your API using Postman or any REST client. Verify each operation works correctly before moving on.

Summary

You have covered the essentials of Spring Boot and JPA. Here is what to take away from this session.

JPA Simplifies Database Interaction

By mapping Java objects to database tables, JPA removes the need for manual SQL and connection management in everyday development.

Spring Data JPA Reduces Code

Extending `JpaRepository` gives you a full suite of CRUD methods instantly — no implementation required.

Focus on Business Logic

With the data layer handled, you can concentrate on what truly matters — solving business problems and delivering value.

👉 🎉 Well done! You now have the foundation to start building real Spring Boot applications with JPA. Keep practising with the hands-on task and explore relationships next.